

defin - Describing what infra you want.
provisioning - creating real infra.

Terraform:

- Terraform is a Infrastructure as code (IAC) tool created by Hashicorp.
- It allows us to define, Provision and manage cloud Infrastructure using declarative Configuration Files. (written in HCL)
- Iac - Everything is written as code, easy to version control in git.
- Multi cloud Support.
- Consistency - Same setup can be deployed across environments (dev, stage, prod)
- Automation - Reduces manual setup and fast, repeatable deployments.
- * We use TF, you want to automate cloud Infrastructure provisioning.
- * you manage multiple environments - dev, test, Prod
- * you want version controlled and repeatable Infrastructure
- * multi-cloud orchestration and setting up CI/CD pipelines.

Workflow:

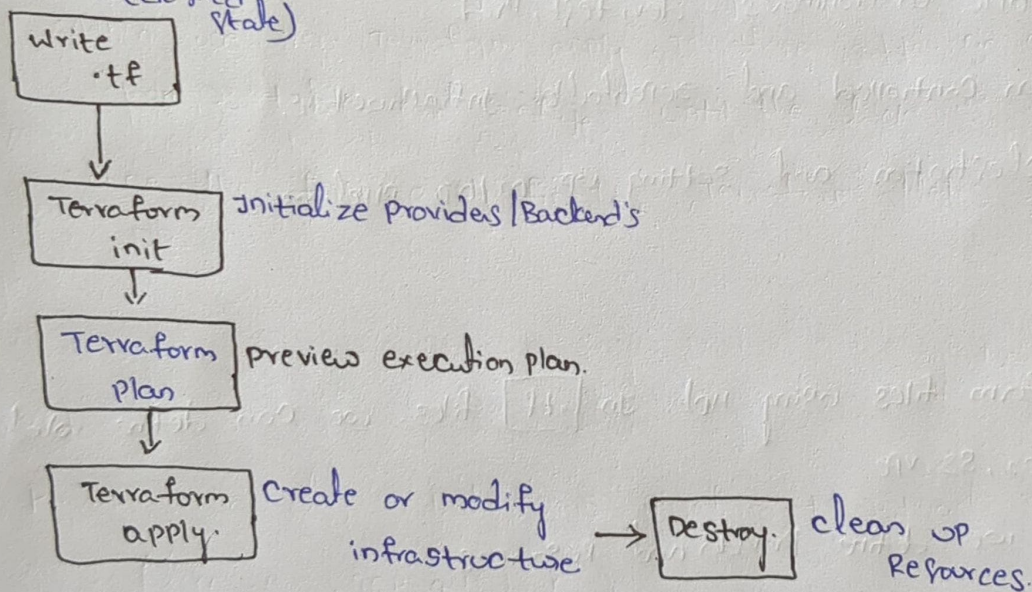
- We write terraform files using HCL in `.tf` files. we can define what we want for eg: EC2, S3, VPC
 - In this file we define Provider and Resources
- **Terraform init** This Command prepares your working directory for Terraform.
 - Downloads the Provider Plugin (AWS, gcp...)
 - It set up the backend (Storing state files in S3)
 - It initializes the Project so Terraform can run properly.
 - Without this Command Terraform won't understand how to talk to aws.
- **Terraform Plan** This Command shows a preview of what Terraform will do.
 - Reads your .tf files and checks the current state file (what's already created then compares it with your desired state (what's in code))

Display's a plan like:

- Resources to be created
- " " changed
- " " Destroyed

- **Terraform apply** This is the execution step. It creates or modifies the resources.
- Show the same plan summary again
 - Ask for Confirmation - yes.
 - Call the AWS API's and create the resources defined in your .tf files.
 - Update the Statefile (terraform.tfstate) with all created resource details (like ID's, IP's etc). { Saves the details of the Resource in a local file called terraform.tfstate }

Define infra (desired state)



Folder after terraform apply:

main.tf - Configuration file

↳ terraform.tfstate

↳ terraform/ - It stores all downloaded provider plugins and module dependencies (This comes after init) need for Terraform to run.

↳ terraform.lock.hcl

↳ This created automatically during terraform init - It locks the versions of the provider plugins used in your project.

* Terraform Statefile:

Terraform statefile is a JSON file that stores the current state of your Infrastructure. (like memory/database) for tracking.

- What resources are created, their current configuration and attributes.
- Without this terraform has no way to know what already exists.

* After Apply Terraform performs these

→ Reads .tf files and state file then compares both and applies only necessary changes to match them.

(here the state file acts as the bridge b/w code and the cloud infra)

* Terraform can store state file in two ways:

① Locally: By default, Terraform stores the state file locally in your project folder. (This is okay for personal or for learning).

- But in Production it may be risky.

→ If multiple people run Terraform their local copies go out of sync.

→ Risk of overwriting.

→ No locking mechanism.

② Remote Backend: Store the state file remotely for collaboration typically on

• S3 (for storage)

• DynamoDB (for state locking)

* If statefile is deleted - Terraform forgets what's been created.

* Statefile is sensitive - It contains secret values (passwords, IP's). Always stores in backend like (S3 with DynamoDB).

* Terraform State lock file :- { This ensures only one process can modify the state file at a time }

State locking is a mechanism that prevents multiple users modifying the same state file at the same time.

→ without locking Terraform state might get corrupted.

→ The state file might lose track.

• Someone else is already modifying the state and Terraform won't proceed until that operation finishes

→ By default state file locking will not support locally Bcz there is no shared storage.

→ Terraform supports locking automatically in Backend (S3 + DynamoDB)

• When we run Terraform apply, Terraform adds an entry (lock) into the DynamoDB table and refuses until the lock is released.

• Once the first apply finishes, Terraform deletes the lock from DynamoDB.

* Providers:-

Providers are plugin that allows Terraform to interact with cloud or service API's (like AWS, GCP, Github, K8's).

→ Terraform downloads all ~~plugins~~ provider plugins during Terraform init.

• Providers authenticates with the service (eg: AWS credentials)

• Translate the Terraform code into real API calls to create/update/delete resources.

Provider supports three main categories of Providers.

① cloud providers

② Infrastructure / Platform Providers - Docker / K8's

③ SaaS providers - Github / Datadog

* Provide Authentication methods:

Eg: AWS

- Access key & secret key
- Env variables
- shared credentials (~/.aws/credentials)
- Jam Role

* Multi Provide (multi cloud / multi-region)

In Terraform multiple Provides allow us to configure and manage resources across different regions / cloud platforms within a single configuration file.

① Multi Region (Same Provide)

Multi Region means using the same cloud provide but deploying resources in different regions.

Eg: creating one EC2 instance in us-east-1 and another in ap-south-1

→ This Terraform achieves this by using Provider aliases

② Multi cloud:

Multi cloud means using different cloud provides (like AWS, GCP) within a single Terraform configuration.

→ Each cloud has its own Provider block with unique credentials and region settings.

Eg: creating EC2 in AWS and Storage in GCP

* Resources:

Resources are infrastructure components that terraform creates, updates, or destroys. These can be anything like EC2, S3, VPC, IAM, DB depending on the provide.

→ These are defined using the resource block in configuration file.

- one Terraform can have multiple Resources.
- Terraform can identify resources using a combination of provider, type and name.

* Data Sources:

Data Sources are allow you to fetch and use existing information from Cloud Provider / other Terraform managed resources without creating new one.

Data Sources in Terraform are used to read existing Infrastructure details things (like AMI ID's, VPC ID's, subnet ID's) and then use those value to create new resources within the same configuration.

* variables:-

variables in Terraform are input parameters that allows you to store values like (region, instance type, AMI ID) and reuse them across configurations.

→ variables make terraform code dynamic, flexible and reusable instead of hardcoding values.

Types:

- ① String variables - store a single text value Eg: t2.micro
- ② Number - stores numeric value Eg: 2, 8080
- ③ List - A sequence of values Eg: us-east-1, us-east-1b
- ④ Map - key value pairs Eg: env = "dev", region = "us-east-1"

→ These values are defined in `variables.tf` and `terraform.tfvars`

① `variables.tf` - defines the input variables like their names, types.

Eg:- variable "region" {

description = "AWS region"

type = string

}

variable "instance-type" {

type = string

② terraform.tfvars - defines the actual value for these variables.

Eg:

```
provider = "aws" {  
    region = var.region  
}
```

```
resource "aws_instance" "web" {  
    ami = 12345...  
    instance_type = var.instance_type  
}
```

This configuration makes cleaner and easier to manage across environments.

Outputs:

In Terraform outputs are used to display or export information about resources after apply configuration.

→ This helps to view key details like instance IP's, ID's, URL's.

→ We use using to see important values

→ To pass/export values to another module or tool like Ansible.

→ It helps to debug or verify infrastructure after deployment.

Eg: main.tf - defining Resources here

output.tf - defining what output should be display.

→ When we run Terraform apply, Terraform shows instance IP, ID's...

Eg:-
→ Let's say you create a EC2 instance and then need it's IP's for Ansible / Jenkins. Instead manually checking AWS, we can get it automatically from Terraform outputs.

* WORK SPACES :-

WORKSPACES In Terraform are used to manage multiple Environments like Dev, Prod, Test using the same Configuration and each workspace keeps its own separate State File.

→ Environment Isolation - keeps own state file, so dev, test environments don't interfere with each other.

→ Same Configuration to deploy resources in different envs.

→ No need to maintain separate folders for each env.

• WORKSPACES are not defined in .tf files.

They are created and managed from the Terraform CLI

Eg: We have Terraform Project ready with main.tf, variables.tf ...

Now in Terminal:

→ Terraform init (Initialize Terraform)

→ terraform workspace dev
" " Prod

{ create a new workspace called dev-Prod }

→ terraform workspace list

→ L terraform workspace select dev
" " " Prod

{ list and switch to environment }

→ Terraform apply Supply infra in that workspace

* Each workspace gets its own state file:

• terraform/

└ terraform.tfstate.d/

└ dev

└ terraform.tfstate

└ prod

└ terraform.tfstate

} completely separate set but the configuration is same.

* Terraform Lifecycle :-

① Terraform init - Initialize your working directory and it prepare Terraform to run.

→ Downloads the required provider plugins and sets the backend where state file will be stored (S3) and creates a lock file to fix provider versions.

• Create's Files/Folders :

• terraform/ - hidden folder contains provider details & backend data.

• terraform.lock.hcl - Records exact provider versions.

{ init is always the first command it sets up providers and initializes backend configuration }

② Terraform validate - validates syntax and structure of all .tf files.
→ Checks syntax, provider versions, and variable references.
→ use validate before ~~apply~~ plan to catch or syntax error's.

③ terraform plan - Show what terraform will do, a preview of infra changes.
→ compares .tf configuration with existing terraform-tfstate. and calculates what needs to be created, updated and destroyed.
→ It doesn't changes anything just it shows what will happen.

④ terraform apply - It creates or updates according to configuration file
→ Reads .tf file then reads the existing terraform.tfstate and "create - update - destroy" on the cloud provider at last update state file with new resource details.

* Files created/updated :

terraform.tfstate

terraform.tfstate.backup (backup of last successful state)

⑤ terraform show :

Displays details of the current state / saved plan.

→ Reads data from the terraform.tfstate file and shows resource attributes like ID's, IP's

⑥ terraform output: shows output value defined in outputs.tf file

→ Reads output from current state file.

⑦ terraform destroy: Deletes all Infrastructure managed by terraform.

→ Reads the state file and removes each resource on provider update the state file after deletion.

{ Terraform lifecycle starts with init → validate → plan → apply → destroy during this it creates files like .terraform/, .terraform.lock.hcl and terraform.tfstate to track providers and infrastructure state }

* terraform.lock.hcl (Dependency lock file)

→ locks provider versions, to ensure everyone uses the same version.

→ creates when terraform init runs.

* terraform.tfstate.backup (Backup file) - Backup last working file (if anything goes wrong on new file backup the old file)

* terraform.tfstate (State file)

→ stores current state of infrastructure. It tracks what's deployed

→ creates ~~empty~~ after apply

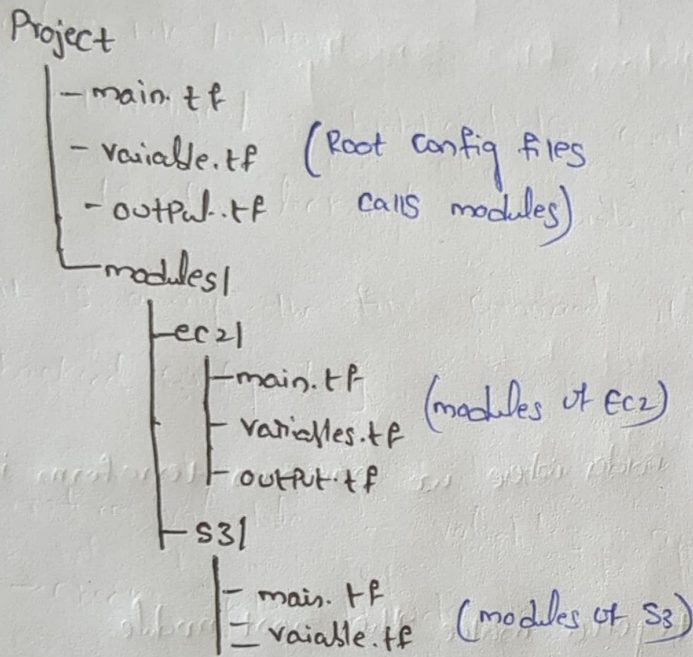
→ terraform.tfstate.lock.info (Temporary lock file)

→ lock file to prevent multiple people editing the same file

Modules:

{ Terraform module is a container for multiple resources that are used together. }

Modules is a way to group related resources together like EC2, VPC, S3 so that we can reuse that same group of resources across multiple environments like dev, test, Prod



→ Each module gets its own subfolder

→ Why: We can configure everything in main.tf (main file) but this is okay for small setup when it comes to large setup we have multiple environments so this way avoids confuses and messy setup.

- Reusability
- If something changes, we can update it once inside the module.
- keeps your project clean and separate logic for EC2, VPC, S3.
- Team collaboration - different teams can maintain their own modules

Eg:- Without module (main.tf)

```
resource "aws_instance" "dev" {  
  ami = "ami-12345"  
  instance-type = "t2.micro"  
}
```

```
resource "aws_instance" "prod" {  
  ami = "ami-1234"  
  instance-type = "t3.micro"  
}
```


With module:

```
module "dev-ec2" {  
  source = ".modules/ec2"  
  ami = "ami-0123"  
  instance-type = "t2.micro"  
}
```

```
module "prod-ec2" {  
  ...  
}
```

* Types of Modules:

① Root module:

This is the main Terraform Project Folder where we can run terraform init, Plan, apply.

→ all .tf files in this folder together make the root module

Eg: main.tf
variables.tf
outputs.tf

② Child modules:

These are reusable modules they're stored inside a subfolder.

Eg: modules/ec2
modules/s3

③ External/Remote modules:

We can also use Public/Private modules from the Terraform Registry or Github.

* Provisioners :

Provisioners are used to execute scripts or command on local machine or remote resource after resource created

→ This helps us to software installation, configuration and file transfer during resource setup.

Eg: I want to create ec2 instance and automatically need to install any package like git/nginx.

```
resource "aws-instance" "example" {
```

```
  ami = "ami-123"
```

```
  instance-type = t2.micro
```

```
  connection {
```

```
    type = "ssh"
```

```
    user = "ubuntu"
```

```
    private_key = file("~/ssh/id_rsa")
```

```
    host = self.Public_IP
```

```
  }
```

```
  provisioner "remote-exec" {
```

```
    inline = [
```

```
      "sudo apt update -y",
```

```
      "sudo apt install nginx -y"
```

```
    ]
```

```
  }
```

```
}
```

→ There types provisioners

- ① local exec - Executes commands on local machine where terraform runs.
- ② remote exec - Executes commands inside resource
- ③ File - Copies files from local to remote resource

* Backend:

Terraform Backend defines where and how terraform stores and manages its state file

→ It allows you to store state file remotely (like S3) instead of locally enabling security and state locking.

→ We can use this Backend in single configuration file like Provider, resource Backend but the Best is maintain files separately.

Eg:

```
terraform {  
  backend "s3" {  
    bucket = "my-terraform-state-bucket"  
    key    = "dev/terraform.tfstate"  
    region = "us-east-1"  
    dynamodb_table = "terraform-lock"  
    encrypt        = true  
  }  
}
```

* Expressions:

Expression in Terraform is a piece of logic that computes a value dynamically instead of writing hardcoded values you can use variables, conditions and references.

Eg: conditional expressions

```
instance_type = var.environment == "Prod" ? "t3.medium" : "t2.micro"
```


* Functions:

Function's are built in functions that help us to manipulate data they make code more powerful and dynamic.

→ String Functions

→ Numeric Functions

→ Collection Functions (Lists/Maps)

→ File Functions.

* Terraform Registry:

It is a Public/Private Repository of pre-built Terraform modules and Providers that we use to quickly set up infrastructure without writing Everything from scratch.

* vault:-

Hashicorp vault is a Secret management tool we can securely store and controls access to sensitive data like passwords, API keys, tokens and certificates.

→ Terraform can integrate with vault to fetch secrets dynamically instead of storing them in code.

* Terraform and vault are Hashicorp tools but serve different purposes where Terraform can create infra and integrates with vault to store secrets in vault.

→ We can integrate vault as a Provider in Terraform to pull credentials securely at runtime.

Complete Project Structure :

Terraform - Project /

- main.tf
- variables.tf
- outputs.tf
- backend.tf
- modules
 - ec2 /
 - main.tf
 - variables.tf
 - outputs.tf
- .terraform / { created by terraform init }
 - providers /
- .terraform.lock.hcl (created by terraform init)
- terraform.tfstate (created by terraform apply)
- terraform.tfstate.backup (Backup of previous state)
- Scripts
 - install-nginx.sh { provisioner setup }
 - deploy-app.sh
- vault (Haproxy vault setup folder)
 - vault.hcl (vault server configuration file)
 - Policies (vault policies)
 - terraform-policy.hcl
 - secrets
 - db-cred.json
 - Scripts (scripts for vault)
 - start-vault.sh
 - unseal-vault.sh